



Haute Ecole de la Province de Liège

Techniques avancées de gestion des données

Bases de données avancées et XML

Leçon 5 : XPath et XSLT

Samuel Hiard

Année académique 2023-2024

Agenda

- Xpath
 - Définition
 - Arbre Xpath
 - Propriétés et fonctions
 - Expressions de cheminement
- XSLT
 - Définition
 - Premier exemple HelloWorld
 - Eléments XSLT
 - Modularité et déploiement

Agenda

- Xpath
 - Définition
 - Arbre Xpath
 - Propriétés et fonctions
 - Expressions de cheminement
- XSLT
 - Définition
 - Premier exemple HelloWorld
 - Éléments XSLT
 - Modularité et déploiement

Xpath : Qu'est-ce que c'est?

- Recommandation du W3C
- Langage permettant de sélectionner des nœuds
- N'est pas un langage autonome : a besoin d'un autre langage
 - Un peu comme les *regexp*
- Syntaxe XPath différente de celle d'XML
 - Mais expressions XPath utilisables comme valeurs d'attributs d'éléments XML
- Evaluation sans effet de bord
 - Pas de variables, pas de mémoire
- Différentes versions : XPath 1.0, XPath 2.0, XPath 3.0, XPath 3.1
 - Nous nous utiliserons des concepts de différentes versions dans ce cours
- XPath 1.0 est utilisé dans XSLT 1.0 et sert de base à XPath 2.0
- Voir <https://www.w3.org/TR/xpath/>

Agenda

- Xpath
 - Définition
 - Arbre Xpath
 - Propriétés et fonctions
 - Expressions de cheminement
- XSLT
 - Définition
 - Premier exemple HelloWorld
 - Éléments XSLT
 - Modularité et déploiement

Arbre XPath

- XPath sélectionne des nœuds en parcourant un arbre
- Arbre XPath $\neq$ Arbre DOM
- Nœuds de l'arbre
 - Document ($\neq$ élément racine, qui est son premier fils)
 - Éléments
 - Attributs
 - Texte
 - Commentaires
 - Instructions de traitement
 - Namespace (seulement en XPath 1.0)
- Sélections possibles sur valeurs
 - Doit pouvoir gérer les types prédéfinis des schémas
 - xsd:string, xsd:integer, xsd:decimal, xsd:date, xsd:time

Ordre du document

- *L'ordre du document* est l'ordre dans lequel on rencontre les objets lorsqu'on parcourt l'arbre du dessus vers le dessous et de gauche à droite.
- Les éléments avant ses enfants, ses attributs et son namespace
- Le namespace avant les attributs
- Les attributs avant les enfants
- L'ordre relatif des attributs et des namespaces est arbitraire (dépend de la plate-forme) mais reste identique dans tout le document
- L'ordre peut être manipulé et intervient de façon implicite dans l'évaluation de certains opérateurs ('/')

Nœud document

- Le nœud racine de l'arbre, pas l'élément racine
- Enfants : l'élément document (racine), les commentaires et les instructions de traitement (en-dehors du document, avant ou après)
- Valeur textuelle: concaténation des valeurs textuelles de tous les nœuds descendants du nœud racine, dans l'ordre du document. Les descendants du nœud sont les enfants du nœud et les descendants des enfants du nœud.
- Nom étendu : aucun

Nœud élément

- Un nœud élément se réfère à un élément
- Enfants : Nœuds éléments, nœuds texte, nœuds commentaires, nœuds instructions de traitement
- Valeur textuelle : concaténation des valeurs textuelles de tous les descendants nœuds texte du nœud élément, dans l'ordre du document
- Nom étendu : partie locale = le nom de l'élément. L'URI du namespace = null s'il n'y a aucun préfixe et pas de namespace par défaut.
- Les nœuds espaces blancs ignorables sont inclus comme enfants

Nœud attribut

- Le parent est l'élément qui le contient...
 - Contrairement au DOM où les attributs ne sont rattachés qu'à leur élément, et n'ont donc pas de parents
- ...mais l'attribut n'est pas son enfant.
- Les attributs par défaut apparaissent dans l'arbre, comme les attributs spécifiés. Un attribut #IMPLIED non spécifié n'a pas de nœud correspondant.
- Valeur textuelle : valeur normalisée, comme spécifié par la recommandation XML. Peut être une chaîne de caractères vide.
- Nom étendu : partie local = nom de l'attribut. URI du namespace = null s'il n'y a pas de préfixe (donc que l'attribut n'est pas dans un NS)
- Les attributs xmlns (pour la déclaration du namespace) n'ont pas de nœuds correspondants.

Nœud namespace

- Déprécié (version XPath 1.0)
- Chaque élément a un ensemble de nœuds pour chaque namespace lié dans la portée de l'élément (préfixé ou par défaut).
- Le préfixe XML est toujours inclus et lié à <http://www.w3.org/XML/1998/namespace>
- L'élément est le parent du nœud namespace, mais les ceux-ci ne sont pas ses enfants
- Valeur textuelle : URI du namespace
- Nom étendu : partie locale = préfixe du namespace (vide si namespace par défaut), URI du namespace = null

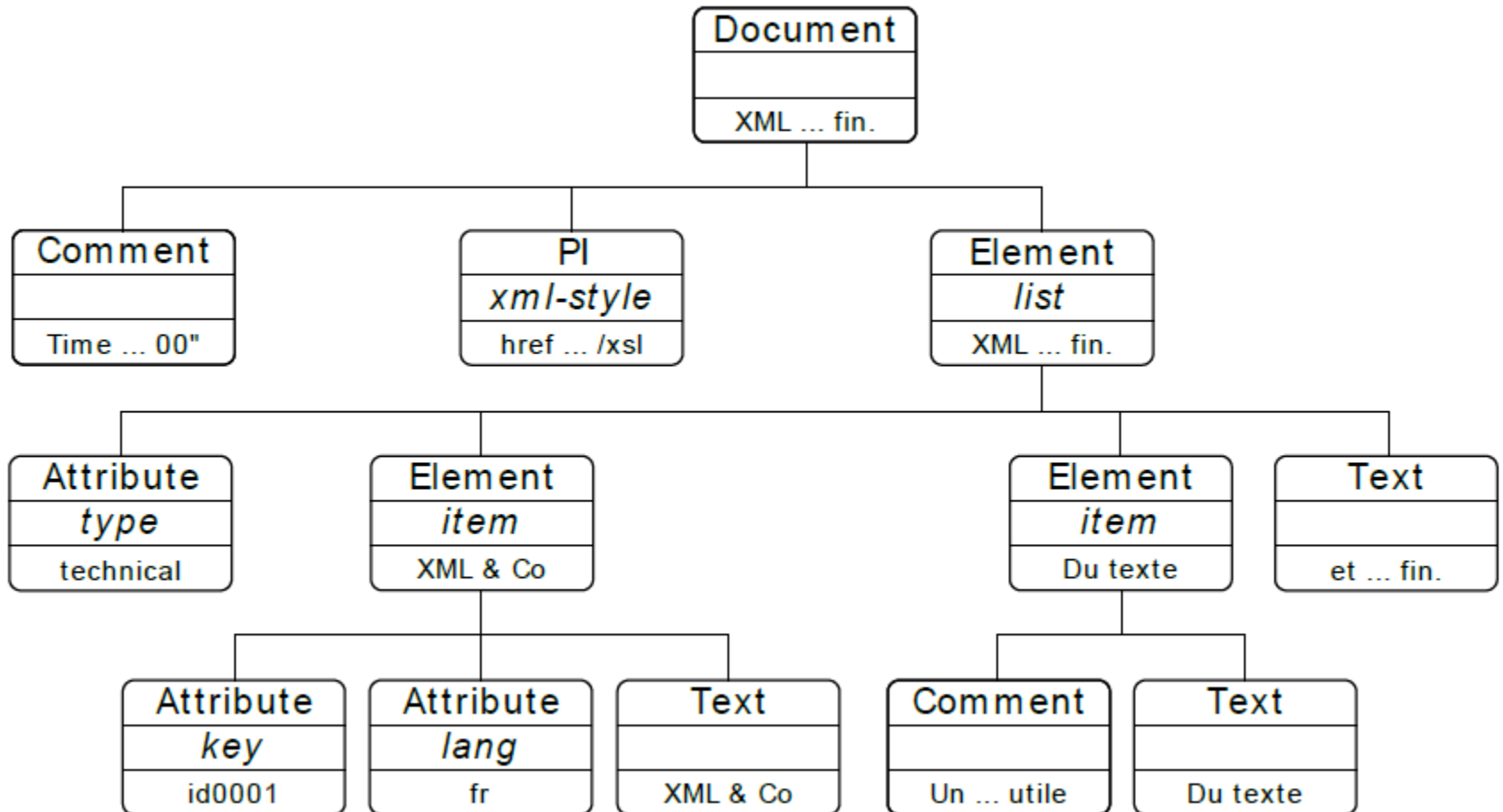
Nœuds textuels, commentaires et instructions de traitement

- Nœuds textuels :
 - Contient des chaînes de caractères. Jamais vide.
 - Autant que possible : pas de nœud textuels contigus. Donc une section CDATA produit un nœud textuel simple.
 - Valeur textuelle : la chaîne de caractères
 - Nom étendu : aucun
- Nœud d'instruction de traitement :
 - Un pour chaque instruction de traitement
 - Valeur textuelle : le texte suivant la cible
 - Nom étendu : partie locale = cible, URI = null
- Nœud commentaire :
 - Un nœud pour chaque commentaire
 - Valeur textuelle : le contenu du commentaire
 - Nom étendu : aucun

Fichier XML d'exemple

```
<?xml version="1.0" encoding="iso-8859-1"?>
<!-- Time-stamp: "tree.xml 14 Feb 2008 09:29:00" -->
<?xml-stylesheet href="tree.xsl" type="text/xsl"?>
<list type="technical">
    <item key="id001" lang="fr">
        XML & Co
    </item>
    <item>
        <!-- Un commentaire inutile -->
        Du texte
    </item>
    et encore du texte à la fin.
</list>
```

L'arbre correspondant



Agenda

- Xpath
 - Définition
 - Arbre Xpath
 - **Propriétés et fonctions**
 - Expressions de cheminement
- XSLT
 - Définition
 - Premier exemple HelloWorld
 - Éléments XSLT
 - Modularité et déploiement

Propriétés des nœuds

- A chaque nœud est associé une liste de propriétés
- Certaines propriétés n'ont pas de sens pour certains types de nœuds, et elles n'ont alors pas de valeur.
- Les propriétés sont :
 - Nom
 - Le nom du nœud. Tout nœud a un nom, sauf les commentaires et les nœuds texte
 - Parent
 - Tout nœud, sauf le nœud racine a un parent. Le parent d'un attribut est l'élément qui le contient
 - Enfants
 - Seul le nœud racine et les éléments peuvent avoir des enfants. Les attributs ne sont pas des enfants des éléments (pas de symétrie!)
 - Valeur textuelle
 - Tout le texte du nœud (y compris celui de ses descendants)
 - Contenu du nœud XML moins les balises, les commentaires et les instructions de traitement
 - Type
 - Seulement si validé
 - Type comme défini par le XSD
 - Valeur typée
 - Seulement si validé
 - Valeur, une fois que le texte a été converti dans le type (ci-dessus)

Fonctions sur les nœuds

- Fonctions permettant d'accéder aux valeurs des propriétés des nœuds
- Type `node()` ? = un nœud. Si pas précisé, nœud courant.
- Fonctions :
 - `xsd:string name(node() ? node)`
 - nom complet (nom namespace + nom local)
 - `xsd:string local-name(node() ? node)`
 - nom local (nom après « : »)
 - `xsd:QName node-name(node() ? node)`
 - nom qualifié (nom complet + namespace URI)
 - `xsd:string string(node() ? node)`
 - valeur textuelle (à ne pas confondre avec la fonction `xsd:string()` qui convertit une valeur en chaîne de caractères.
 - `xsd:anyAtomicType* data(node() * list)`
 - valeurs typées de la liste
 - `node() root(node() ? node)`
 - racine de l'arbre
 - `node() * id(xsd:string s)`
 - liste des nœuds dont la valeur de l'attribut de type ID est un des noms de la chaîne s

Fonctions (suite)

- `xsd:anyURI base-uri(node() ? node)`
 - URI de base du noeud
- `xsd:anyURI document-uri(node() ? node)`
 - URI de base du document contenant le noeud
- `xsd:anyURI resolve-uri(xsd:string? url, xsd:string? base)`
 - combine l'URI uri avec l'URI de base base
- `xsd:string* namespace-uri(node() ? node)`
 - URI du namespace
- `xsd:string* in-scope-prefixes(node() node)`
 - liste des préfixes associés au namespace du nœud
- `xsd:string namespace-uri-for-prefix(xsd:string prefix, node() node)`
 - URI associé au namespace associé au préfixe dans le nœud

Modèle de données

- Une *valeur* est soit un nœud, soit une valeur atomique.
- Le type de valeur le plus répandu est la liste
 - Elle ne contient que des valeurs (pas des listes imbriquées)
 - Ses éléments sont numérotés à partir de 1
 - On la construit avec l'opérateur virgule (« , »)
 - Toute valeur peut être considérée comme une liste de longueur 1
- Les valeurs atomiques sont les types primitifs
 - xsd:string
 - Chaîne de caractères : 'string' ou "string"
 - xsd:boolean
 - Booléen : true() et false()
 - xsd:decimal
 - Nombre décimal : 3.14
 - xsd:float et xsd:double
 - Nombre flottant en simple et double précision
 - xsd:integer
 - Entier : 42
 - xsd:duration, xsd:yearMonthDuration et xsd:dayTimeDuration
 - Durée : P6Y4M2DT11H22M44S de 6 ans, 4 mois, 2 jours, 11 heures 22 minutes et 44 secondes.
 - xsd:date, xsd:time et xsd:dateTime
 - Date et heure : 2009-03-02T11:44:22
 - xsd:anyURI
 - URI : <http://www.omega-one.org/~carton/>
 - ...

Typage

- Système de typage pour définir les paramètres et les valeurs de retour des fonctions
- Défini dans un arbre, dont la racine est item()
 - Tout type est une spécialisation de item()
- Types prédéfinis
 - Voir slide précédent
- Types de nœuds
 - node() : nœud générique, dont dérive :
 - document-node()
 - element()
 - attribute()
 - text()
 - comment()
 - processing-instruction()

Agenda

- Xpath
 - Définition
 - Arbre Xpath
 - Propriétés et fonctions
 - Expressions de cheminement
- XSLT
 - Définition
 - Premier exemple HelloWorld
 - Éléments XSLT
 - Modularité et déploiement

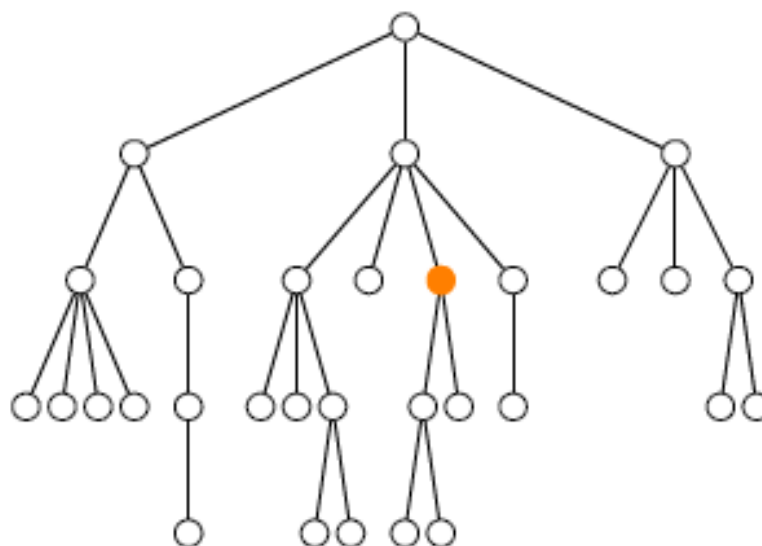
Expressions de cheminement

- Pour répondre au besoin premier de XPath : sélectionner des nœuds se trouvant dans l'arbre
- 2 syntaxes : explicite et abrégée
- Syntaxe explicite
 - Expression sous la forme `axe::type`
 - Axe : applique un premier filtre sur les nœuds par rapport au nœud courant
 - Type : type de nœud (ou son nom)
 - Possibilité d'appliquer également d'autres filtres
- Syntaxe abrégée
 - Souvent rencontrée
 - Voir plus loin

Axes

self

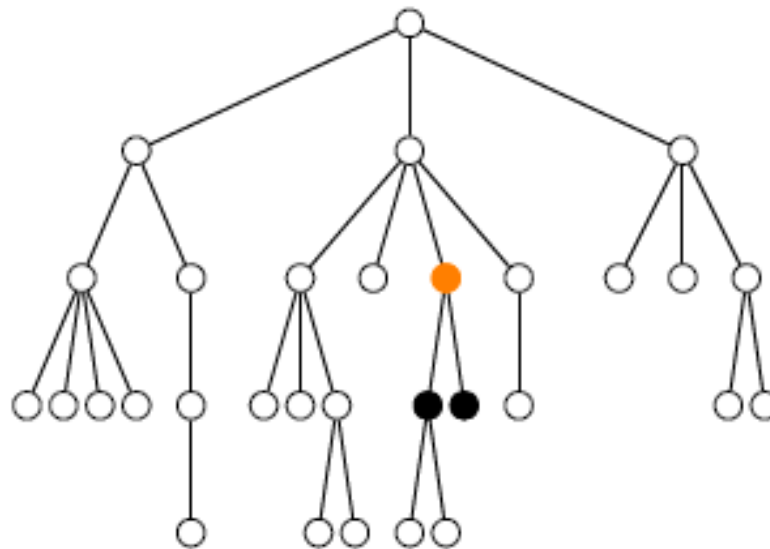
Le nœud lui-même (égalité)



Axes (2)

child

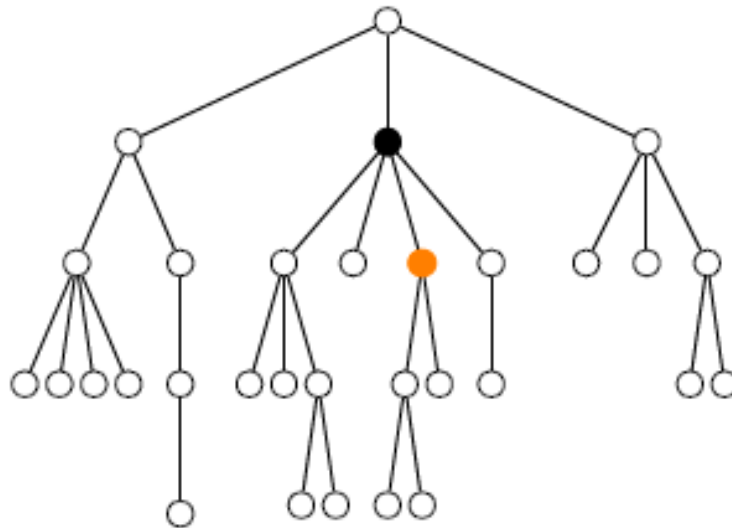
Enfant direct



Axes (3)

parent

Parent



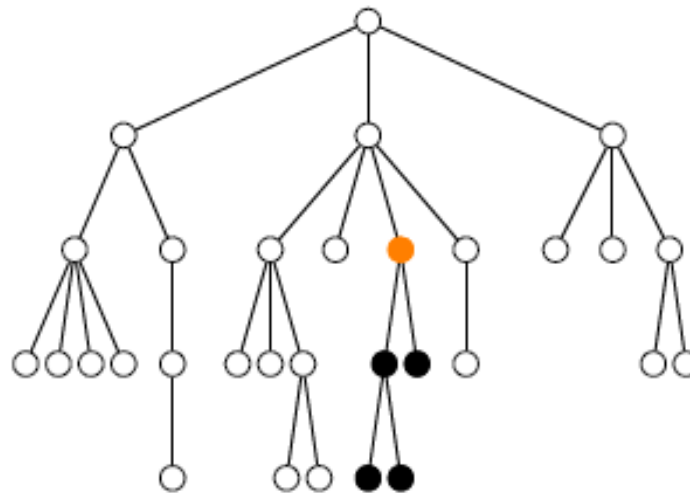
attribute

Attribut du nœud

Axes (4)

descendant

Descendant strict



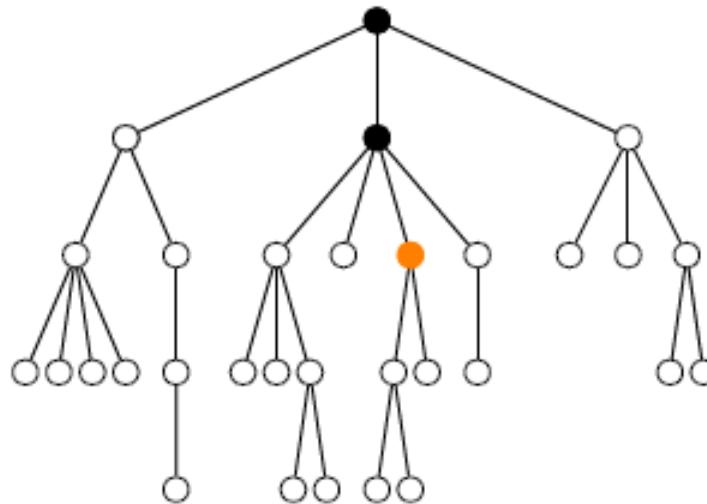
descendant-or-self

Descendant ou le nœud lui-même

Axes (5)

ancestor

Ancêtre strict



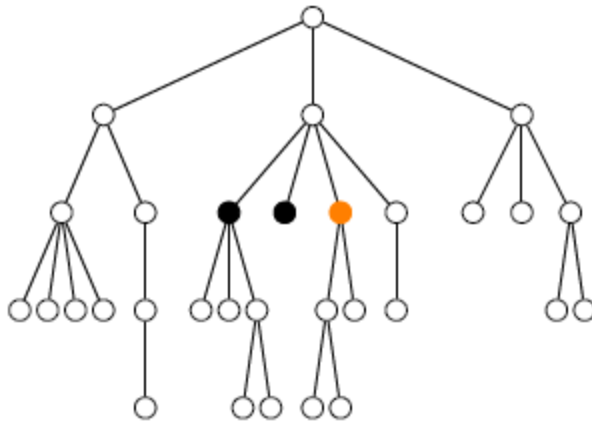
ancestor-or-self

Ancêtre ou le nœud lui-même

Axes (6)

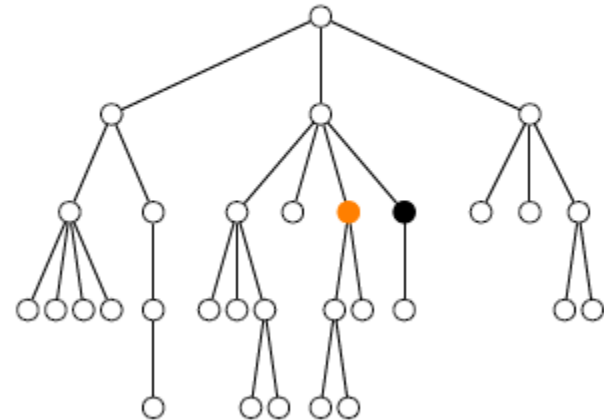
preceding-sibling

Frère gauche (enfant du même parent)



following-sibling

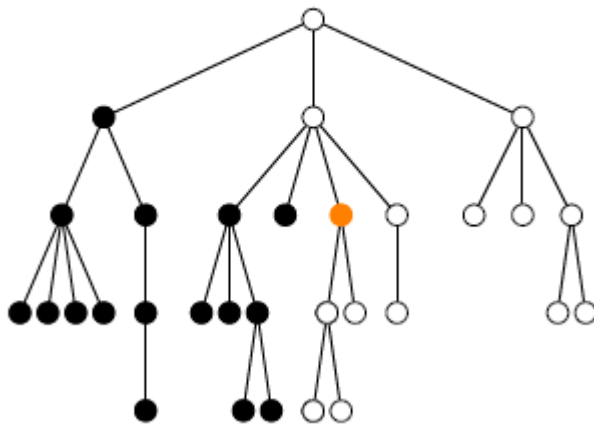
Frère droit (enfant du même parent)



Axes (7)

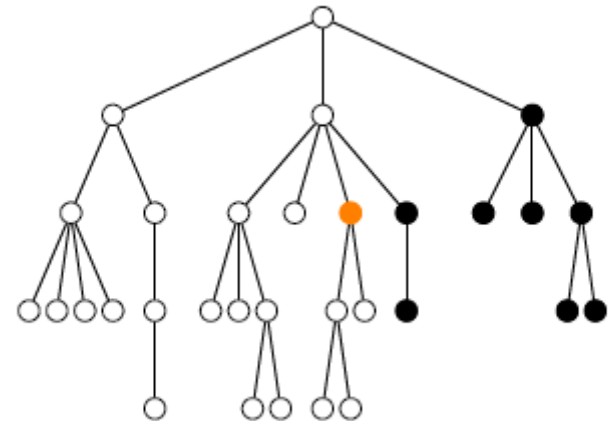
preceding

À gauche



following

À droite



Types

- *
 - tous les éléments ou tous les attributs suivant l'axe utilisé
- ns : *
 - tous les éléments ou tous les attributs (suivant l'axe) dans l'espace de noms associé au préfixe ns
- * : local
 - tous les éléments ou tous les attributs (suivant l'axe) de nom local local
- name
 - les noeuds de nom name (éléments ou attributs suivant l'axe utilisé)
- node ()
 - tous les noeuds
- text ()
 - tous les noeuds textuels
- comment ()
 - tous les commentaires
- processing-instruction() ou processing-instruction(name)
 - toutes les instructions de traitement ou les instructions de traitement de nom name

Exemples

`child::*`

- tous les éléments qui sont enfants du noeud courant

`attribute::*`

- tous les attributs du noeud courant

`attribute::id`

- attribut id du noeud courant

`child::node()`

- tous les enfants du noeud courant

`child::text`

- tous les éléments de nom text qui sont enfants du noeud courant

`child::text()`

- tous les noeuds textuels qui sont enfants du noeud courant

`descendant::comment()`

- tous les commentaires qui sont descendants du noeud courant, c'est-à-dire contenus dans le noeud courant

`following::processing-instruction()`

- toutes les instructions de traitement qui suivent le noeud courant.

`child::processing-instruction('xml-stylesheet')`

- toutes les instructions de traitement de nom xml-stylesheet qui sont enfants du noeud courant.

Filtres supplémentaires

- Il est possible de restreindre encore le résultat de la sélection avec des filtres (dans des crochets [])
- Il existe 27 fonctions permettant de filtrer
 - Nous ne les verrons pas toutes ici
- Voici, à la place, quelques exemples explicites

```
child::*[position() < 4]
```

- les trois premiers éléments enfants du noeud courant

```
attribute::*[name() != 'id']
```

- tous les attributs autres que id. Cette expression peut aussi être écrite @* except @id.

```
child::node()[position() = 4]
```

- le quatrième enfant du noeud courant

```
child::section[position() = last()]
```

- dernier enfant section du noeud courant

```
descendant::item[attribute::id]
```

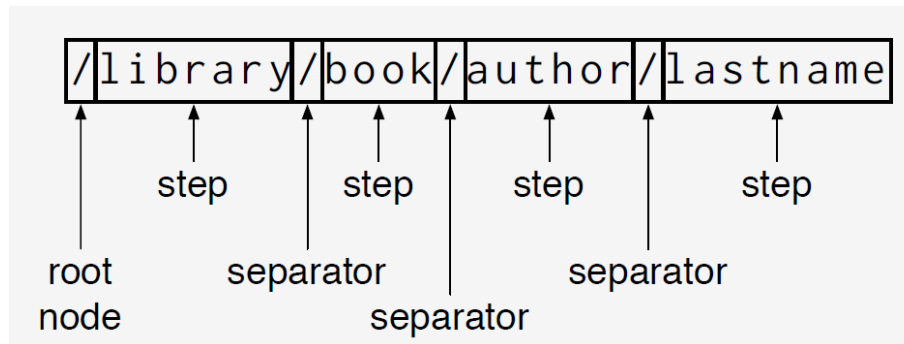
- tous les descendants du noeud courant qui sont un élément item ayant un attribut id

```
chapter[count(child::section) > 1]
```

- élément chapter ayant au moins deux éléments section comme enfants.

Composition de chemins

- Comme en shell Unix, nous avons l'opérateur « / »
- Celui-ci nous permet de composer des chemins
- Sa syntaxe est un peu différente, il fonctionne comme suit :
 - Soit `expr1/expr2` une expression
 - `expr1` est d'abord évaluée et retourne une liste de nœuds
 - Pour **chacun de ces nœuds**, j'évalue `expr2`
 - J'ai donc potentiellement plusieurs groupes pour `expr2`
 - Je les concatène, et ceci donne ma valeur de retour
- Extensible à plusieurs sous-chemins



Quelques exemples

`self::node() / child::*`

- tous les éléments qui sont enfants du noeud courant

`child::* / child::*`

- tous les éléments qui sont des enfants des enfants du noeud courant

`child::p / child::em`

- tous les éléments em enfants d'un élément p enfant du noeud courant

`child::* / attribute::*`

- tous les attributs des éléments qui sont enfants du noeud courant

`parent::* / parent::*`

- le parent du parent du noeud courant

`parent::* / child::*`

- tous éléments qui sont frères du noeud courant, y compris le noeud courant

`child::* / parent::*`

- le noeud courant s'il contient au moins un élément ou aucun noeud sinon

`descendant::p / child::em`

- tous les éléments em qui sont enfants d'un élément p descendant du noeud courant

Syntaxe abrégée

- Il est possible d'utiliser une syntaxe abrégée

Abbréviation	Traduction
.	self::node() (déprécié; XPath 1.0)
..	parent::node()
/	nœud racine
//	/descendant-or-self::node()/
@	attribute::
[<i>n</i>]	[position()= <i>n</i>]

- A noter que // lui-même n'est pas valide
 - Termine par un séparateur sans rien derrière
 - Par exemple, //section[1] et /descendant-or-self::section[1] peuvent ne pas donner le même résultat

Agenda

- Xpath
 - Définition
 - Arbre Xpath
 - Propriétés et fonctions
 - Expressions de cheminement
- XSLT
 - Définition
 - Premier exemple HelloWorld
 - Éléments XSLT
 - Modularité et déploiement

XSLT : Qu'est-ce que c'est?

- XSLT signifie eXtensible Stylesheet Language: Transformations
- XSLT 1.0 est une recommandation W3C
- Depuis le 16/11/1999

<http://www.w3.org/TR/xslt>

A l'origine, XSLT faisait partie de XSL (eXtensible Stylesheet Language) dont le but était de définir le format et la présentation des documents XML. Mais, plus tard, XSL a été décomposé en deux parties, plutôt indépendantes, généralement utilisées en séquence:

- D'abord, une transformation structurelle avec XSLT
 - Ce qu'on va voir ici
- Ensuite, un processus de formatage (principalement pour l'impression) avec XSL-FO (XSL Formatting Objects).
 - Dans la littérature, on y fait également référence sous le nom XSL, d'où la confusion possible.

Un peu de terminologie

- CSS = Cascaded style sheet
 - HTML utilise des balises prédéfinies, dont la signification et la manière de les afficher est bien comprise.
 - On utilise alors CSS pour mettre en forme des documents HTML.
- XSL = Style Sheets for XML
 - XML n'utilise pas de balises prédéfinies, et donc leur signification n'est pas bien comprise.
 - Un élément <table> pourrait tout aussi bien être un tableau HTML, la table de la salle à manger, ou n'importe quoi d'autre, et les navigateurs ne savent pas quoi faire avec!
 - Donc, bien qu'il ne soit pas limité à cet usage, XSL peut d'écrire comment les éléments XML doivent être affichés.
- Pour résumer :
 - XSLT – un langage pour transformer des documents XML
 - XPath – un langage pour naviguer dans des documents XML
 - XSL-FO – Un langage pour formater des documents XML (discontinué en 2013)
 - XQuery – Un langage pour effectuer des requêtes sur un document XML

Définition

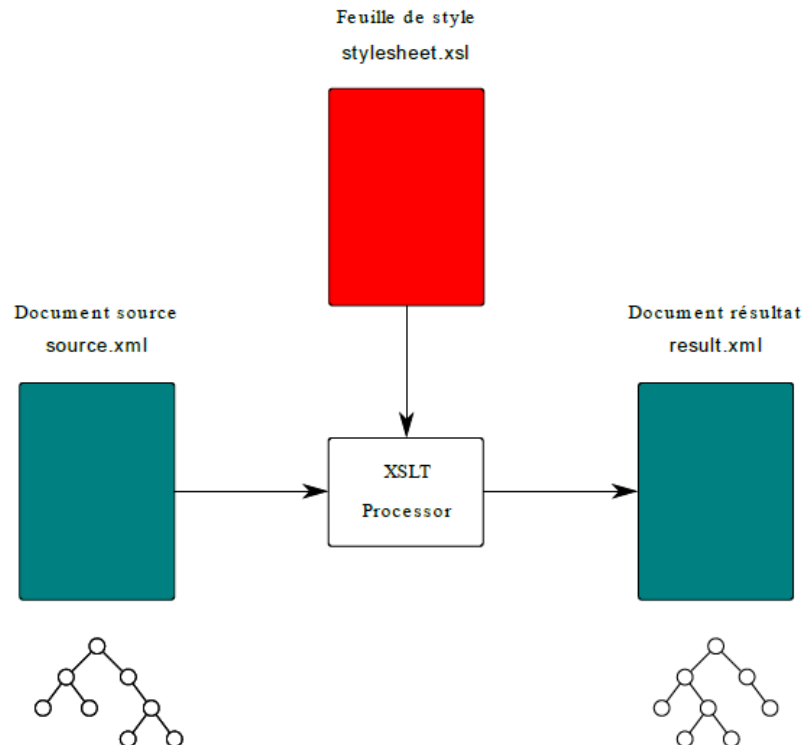
- XSLT est un vocabulaire XML
- **Objectif principal** : transformer un document XML dans un autre document (XML, texte brut, XHTML). C'est une transformation de texte en texte, il n'y a pas de données binaires.
- **Comment ?** En utilisant un ensemble de règles inclues dans des documents appelés « feuilles de style » (stylesheets)
- **Pourquoi ?** XML est utilisé pour stocker des données brutes dans un format spécifique. Parfois, nous avons besoin d'un autre format, pour présenter les données (écran, audio, ...), pour envoyer les données vers un autre système, ...

Arbre XSLT

- XSLT opère sur des informations représentées sous forme d'arbre abstrait.
- Quelles informations? Le document source, le document cible, et la feuille de style. Les 3 sont représentés sous forme d'arbre.
- Ces représentations sont exprimées en utilisant le modèle de données.
- Il ressemble très fort à celui de XPath 1.0, avec quelques différences
- Les commentaires et les instructions de traitement ne font pas partie de l'arbre représentant la feuille de style.

Processus de transformation

- Le processeur XSLT effectue le processus de transformation.
- Il a deux entrées et une sortie.
- Il prend l'arbre source depuis le document source, et l'arbre de la feuille de style depuis le document feuille de style
- Il produit un arbre résultat et le sérialise vers un document résultat.



Agenda

- Xpath
 - Définition
 - Arbre Xpath
 - Propriétés et fonctions
 - Expressions de cheminement
- XSLT
 - Définition
 - Premier exemple HelloWorld
 - Éléments XSLT
 - Modularité et déploiement

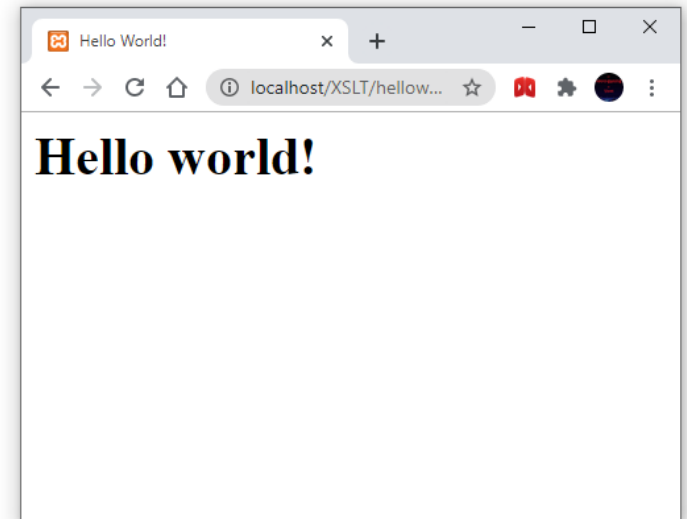
Hello World XSLT

- Dans le fichier XML

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet href="./helloworld.xsl" type="text/xsl" ?>
<rien />
```

- Dans le fichier XSLT

```
<?xml version="1.0" encoding="iso-8859-1"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
  <html>
    <head>
      <title>Hello World!</title>
    </head>
    <body>
      <h1>Hello world!</h1>
    </body>
  </html>
</xsl:template>
</xsl:stylesheet>
```



Explications

XML

```
<?xml version="1.0" encoding="UTF-8"?>
```

- Comme tout fichier XML, commence par une en-tête

```
<?xml-stylesheet href="./helloworld.xsl"  
type="text/xsl" ?>
```

- Fait référence au fichier XSLT

```
<rien />
```

- Dans cet exemple, ne sert pas à grand-chose, mais un document XML doit avoir un élément racine...

Explications (2)

XSLT

```
<?xml version="1.0" encoding="iso-8859-1"?>
```

- C'est aussi un fichier XML, donc commence par une en-tête

```
<xsl:stylesheet version="1.0"
```

```
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

- Indique qu'il s'agit d'une feuille de style XSLT, le namespace et la version

```
<xsl:template match="/">
```

- Élément **template** qui définit quoi faire. L'attribut **match** précise quels nœuds sont affectés
- Ici, « / » veut dire qu'on sélectionne le nœud racine
- Il s'agit d'une expression XPath !

```
<html> ... </html>
```

- Contenu à remplacer lorsque le nœud a été rencontré
- Notez qu'ici, le fichier XML n'est pas du tout pris en compte!

```
</xsl:template> </xsl:stylesheet>
```

- Fermeture des tags, nous sommes en XML

Agenda

- Xpath
 - Définition
 - Arbre Xpath
 - Propriétés et fonctions
 - Expressions de cheminement
- XSLT
 - Définition
 - Premier exemple HelloWorld
 - **Éléments XSLT**
 - Modularité et déploiement

Fichier XML d'exemple

- Pour illustrer la suite, nous allons utiliser le fichier XML suivant :

```
<catalog>
  <cd>
    <title>Empire Burlesque</title>
    <artist>Bob Dylan</artist>
    <price>10.90</price>
  </cd>
  <cd>
    <title>Hide your heart</title>
    <artist>Bonnie Tyler</artist>
    <price>9.90</price>
  </cd>
  <cd>
    <title>Greatest Hits</title>
    <artist>Dolly Parton</artist>
    <price>9.90</price>
  </cd>
</catalog>
```

Value-of

- Permet d'extraire le contenu d'un élément et l'intégrer dans l'output

```
<xsl:template match="/">
  <html>
  <body>
    <h2>My CD Collection</h2>
    <table border="1">
      <tr bgcolor="#9acd32">
        <th>Title</th>
        <th>Artist</th>
      </tr>
      <tr>
        <td><xsl:value-of select="catalog/cd/title"/></td>
        <td><xsl:value-of select="catalog/cd/artist"/></td>
      </tr>
    </table>
  </body>
</html>
</xsl:template>
```

My CD Collection

Title	Artist
Empire Burlesque	Bob Dylan

For-each

- Récupère l'ensemble des valeurs du nodeset

```
<xsl:template match="/">
  <html>
  <body>
  <h2>My CD Collection</h2>
  <table border="1">
    <tr bgcolor="#9acd32">
      <th>Title</th>
      <th>Artist</th>
    </tr>
    <xsl:for-each select="catalog/cd">
      <tr>
        <td><xsl:value-of select="title"/></td>
        <td><xsl:value-of select="artist"/></td>
      </tr>
    </xsl:for-each>
  </table>
  </body>
  </html>
</xsl:template>
```

My CD Collection

Title	Artist
Empire Burlesque	Bob Dylan
Hide your heart	Bonnie Tyler
Greatest Hits	Dolly Parton

Filtre supplémentaire

- Comme pour les expressions XPath

```
<xsl:template match="/">
  <html>
  <body>
  <h2>My CD Collection</h2>
  <table border="1">
    <tr bgcolor="#9acd32">
      <th>Title</th>
      <th>Artist</th>
    </tr>
    <xsl:for-each select="catalog/cd[position() <= 3]">
      <tr>
        <td><xsl:value-of select="title"/></td>
        <td><xsl:value-of select="artist"/></td>
      </tr>
    </xsl:for-each>
  </table>
  </body>
  </html>
</xsl:template>
```

My CD Collection

Title	Artist
Empire Burlesque	Bob Dylan
Hide your heart	Bonnie Tyler

Sort

- Permet de trier les éléments

```
<xsl:template match="/">
  <html>
  <body>
  <h2>My CD Collection</h2>
  <table border="1">
    <tr bgcolor="#9acd32">
      <th>Title</th>
      <th>Artist</th>
    </tr>
    <xsl:for-each select="catalog/cd">
      <xsl:sort select="title"/>
      <tr>
        <td><xsl:value-of select="title"/></td>
        <td><xsl:value-of select="artist"/></td>
      </tr>
    </xsl:for-each>
  </table>
  </body>
  </html>
</xsl:template>
```

My CD Collection

Title	Artist
Empire Burlesque	Bob Dylan
Greatest Hits	Dolly Parton
Hide your heart	Bonnie Tyler

if

- N'affiche quelque chose que si une condition est vraie.
- Syntaxe :

```
<xsl:if test="expression">  
    ...une valeur à afficher si l'expression est vraie...  
</xsl:if>
```

- Peut se placer dans un for-each

```
<xsl:for-each select="catalog/cd">  
    <xsl:if test="price>10">  
        <tr>  
            <td><xsl:value-of select="title"/></td>  
            <td><xsl:value-of select="artist"/></td>  
            <td><xsl:value-of select="price"/></td>  
        </tr>  
    </xsl:if>  
</xsl:for-each>
```

My CD Collection

Title	Artist	Price
Empire Burlesque	Bob Dylan	10.90

Choose

- Permet d'exprimer plusieurs tests conditionnels
- Syntaxe

```
<xsl:choose>
  <xsl:when test="expression">
    ... quelque chose à afficher ...
  </xsl:when>
  <xsl:otherwise>
    ... autre chose à afficher ....
  </xsl:otherwise>
</xsl:choose>
```

Choose (2)

```
<xsl:for-each select="catalog/cd">
<tr>
  <td><xsl:value-of select="title"/></td>
  <xsl:choose>
    <xsl:when test="price > 10">
      <td bgcolor="#ff00ff">
        <xsl:value-of select="artist"/></td>
      </xsl:when>
      <xsl:otherwise>
        <td><xsl:value-of select="artist"/></td>
      </xsl:otherwise>
    </xsl:choose>
  </tr>
</xsl:for-each>
```

My CD Collection

Title	Artist
Empire Burlesque	Bob Dylan
Hide your heart	Bonnie Tyler
Greatest Hits	Dolly Parton

Agenda

- Xpath
 - Définition
 - Arbre Xpath
 - Propriétés et fonctions
 - Expressions de cheminement
- XSLT
 - Définition
 - Premier exemple HelloWorld
 - Éléments XSLT
 - Modularité et déploiement

Apply-templates

- Element permettant d'appliquer les templates à des nœuds
- Le premier template est automatiquement appliqué
 - Un peu comme un appel de fonction
- Les templates sont appliqués aux nœuds
 - Tous les templates sont considérés, et on filtre par rapport au *match*
 - S'il y a conflit, on regarde la priorité, qui est définie en fonction
 - De l'ordre d'importation des fichiers XSL
 - D'une valeur par défaut attribuée en fonction de l'expression Xpath
 - De la priorité explicite spécifiée
- Si deux templates avec la même priorité doivent s'appliquer au même nœud, c'est une erreur
- Possibilité d'utiliser call-template avec un nom de template pour différencier.

Example apply-templates

```
<xsl:template match="/">
  <html>
    <body>
      <h2>My CD Collection</h2>
      <xsl:apply-templates/>
    </body>
  </html>
</xsl:template>
```

```
<xsl:template match="cd">
  <p>
    <xsl:apply-templates select="title"/>
    <xsl:apply-templates select="artist"/>
  </p>
</xsl:template>
```

```
<xsl:template match="title">
  Title: <span style="color:#ff0000">
    <xsl:value-of select="."/></span>
  <br />
</xsl:template>
```

```
<xsl:template match="artist">
  Artist: <span style="color:#00ff00">
    <xsl:value-of select="."/></span>
  <br />
</xsl:template>
```

Lien entre XSLT et XML

- Jusqu'ici, le fichier XSLT était référencé dans le document XML
- Ceci implique que le navigateur est *XSLT-aware*, ce qui n'est pas toujours le cas!
- Deux solutions :
 - Lien côté client avec AJAX
 - Lien côté serveur avec PHP
- Nous ne verrons aucune de ces deux technologies dans ce cours, mais pour info :
 - https://www.w3schools.com/xml/xsl_client.asp
 - https://www.w3schools.com/xml/xsl_server.asp